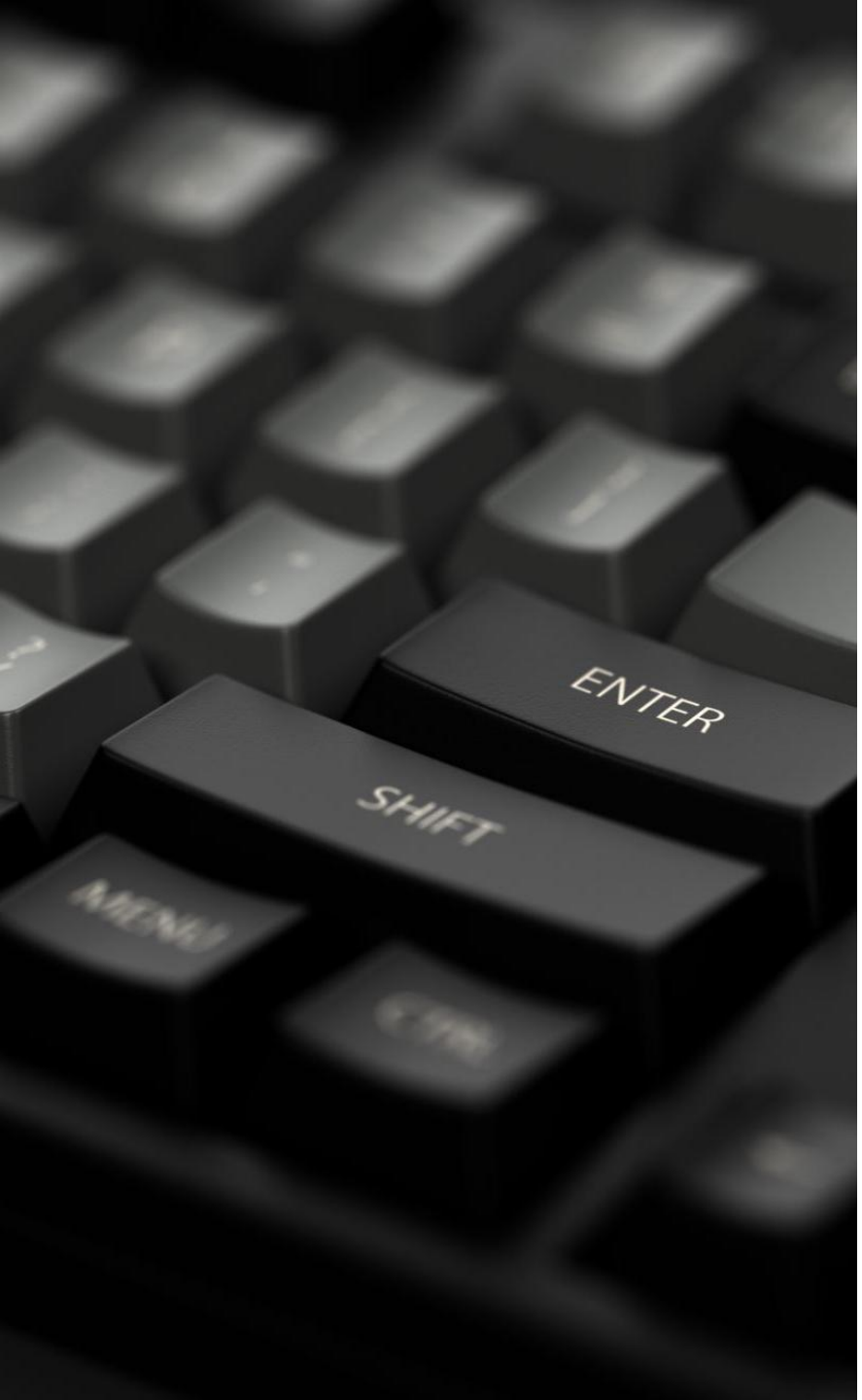
A gamified introduction to Python Programming

Lecture 13
Importing and Project Management

Matthieu DE MARI – Singapore University of Technology and Design



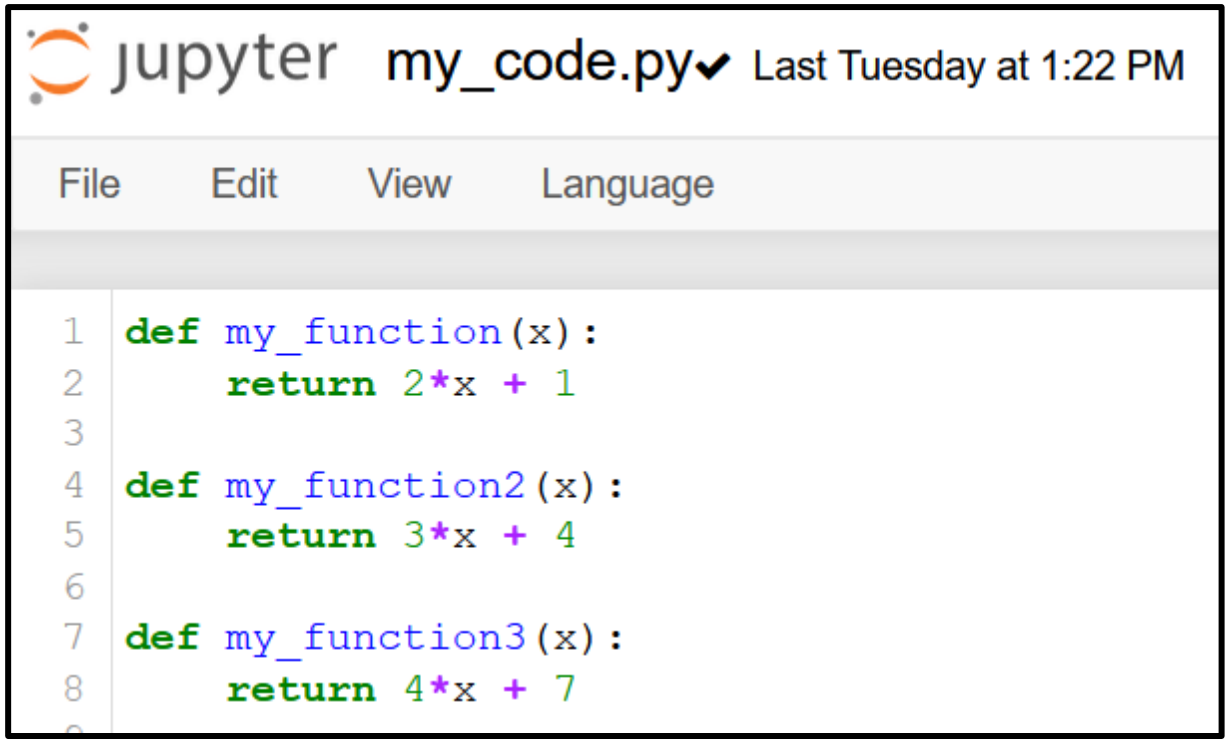
Outline (Chapter 13)

- What is the **import procedure** doing exactly?
- How to organize your different codes for better **project organizing**?
- **Our first mini-project**.

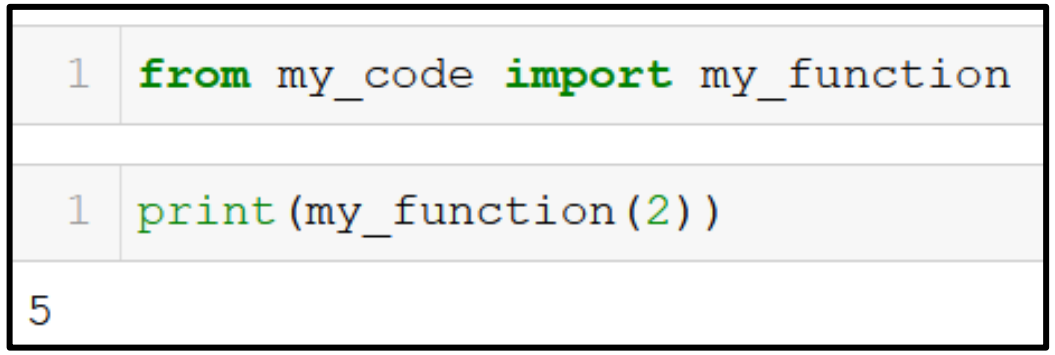
The import procedure

- The import procedure is used to **import functions defined in external .py files.**
- To demonstrate, we have defined a **my_code.py** file with three functions.
- We can then import one of these functions in our Notebook, by using the **from … import …** command.

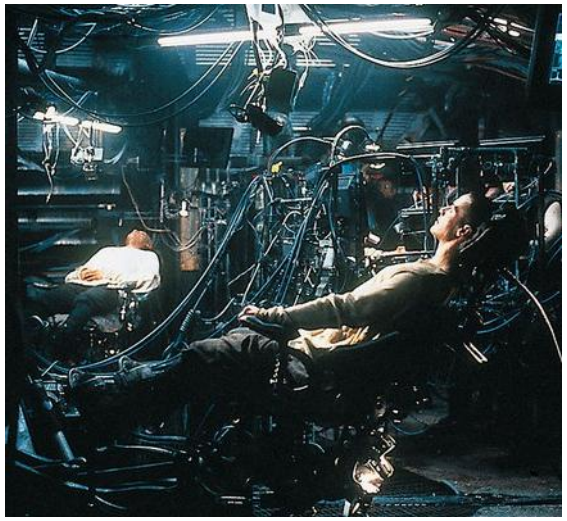
my_code.py file



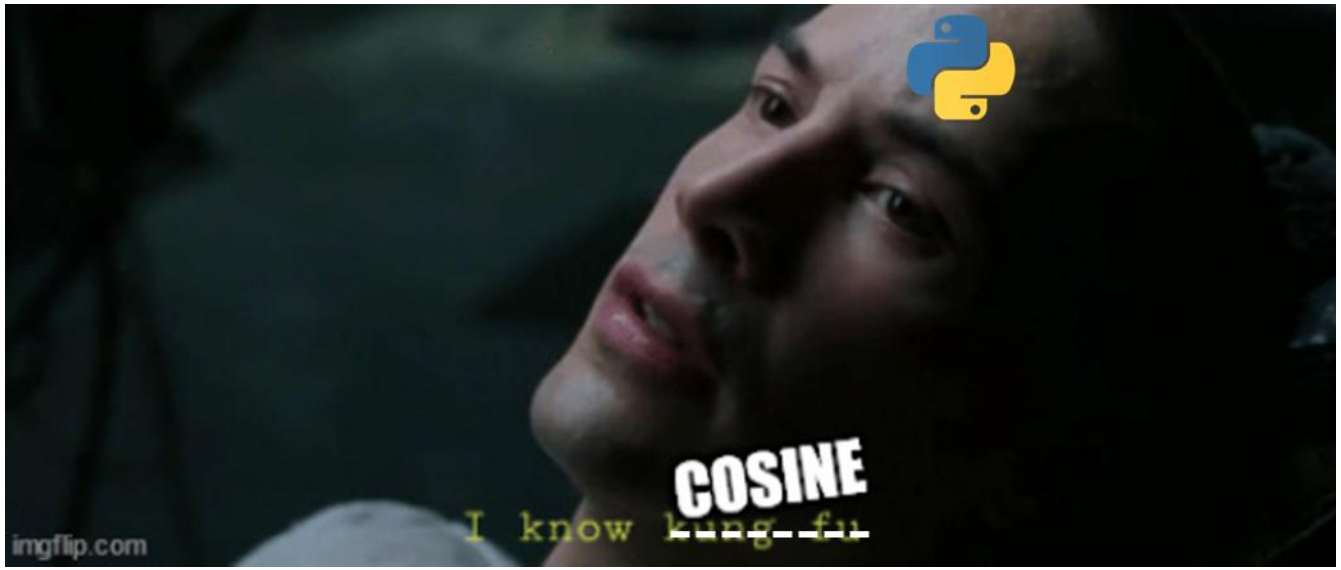
In our notebook



The import procedure



Operator:
Cosine function coming right up!




```
1 print(cos(0))
```

Importing as

If needed, we can **import** and **rename** a function by using the **as** keyword after an **import**.

- The whole command then reads **from … import … as …**

- **Note:** if you rename the function, its calling name changes to the alias you specified.

my_code.py file

jupyter my_code.py ✔ Last Tuesday at 1:22 PM

File Edit View Language

```
1  def my_function(x):
2      return 2*x + 1
3
4  def my_function2(x):
5      return 3*x + 4
6
7  def my_function3(x):
8      return 4*x + 7
```

In our notebook

```
1  from my_code import my_function2 as custom_name
```

```
1  print(my_function2(2))
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-4-20b74e0a7273> in <module>
----> 1 print(my_function2(2))

NameError: name 'my_function2' is not defined
```

```
1  print(custom_name(2))
```

```
10
```

Importing several functions

If needed, you can import **multiple functions at once**.

- Simply use **commas (,) symbols** to separate the different functions names.
- Or, you can also **import all functions** at once, by simply entering *.

(This is considered malpractice, however, you should just import what you need!)

my_code.py file

jupyter my_code.py ✔ Last Tuesday at 1:22 PM

File Edit View Language

```
1  def my_function(x):
2      return 2*x + 1
3
4  def my_function2(x):
5      return 3*x + 4
6
7  def my_function3(x):
8      return 4*x + 7
```

In our notebook

```
1  from my_code import my_function, my_function2
```

```
1  from my_code import *
```

```
1  print(my_function3(2))
```

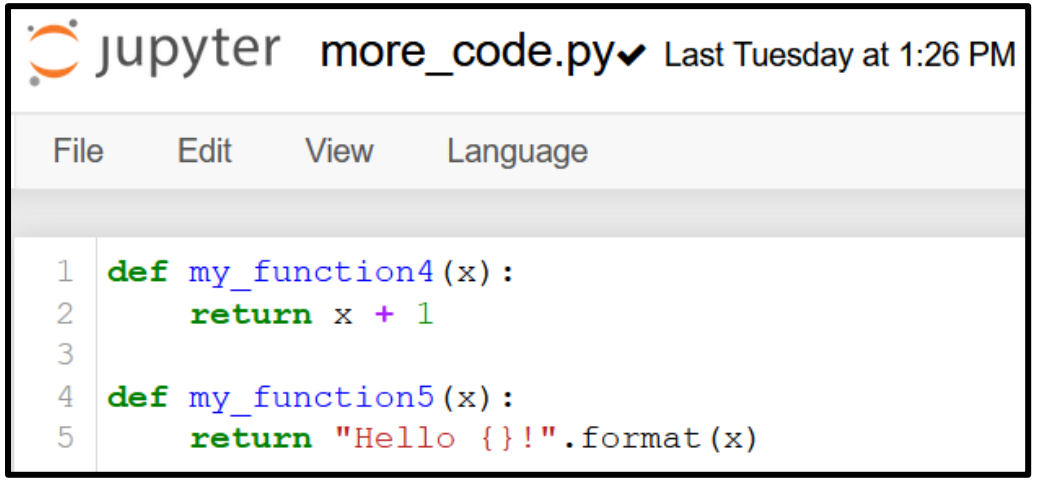
```
15
```

Importing an entire module

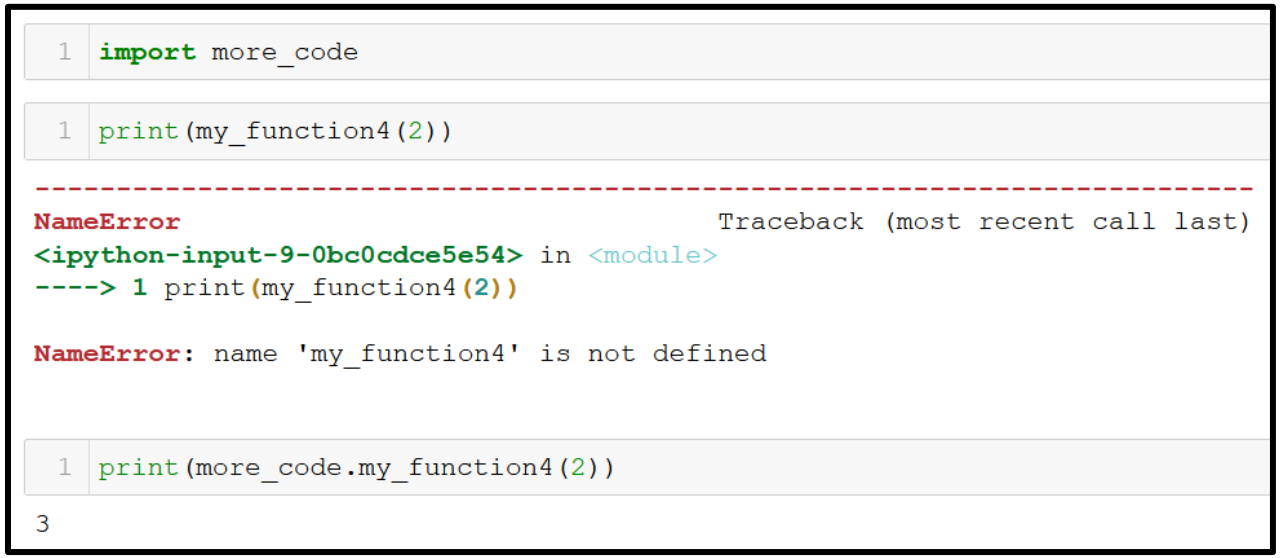
You can also import a whole file or library as a **module.**

- To do so, simply start with **import** instead of **from**.
- By doing so, you import all the functions, but **they have to be called using the module name and the dot operator** (.).
- This can make the code more readable, but is also a bit more inconvenient (your choice!).

more_code.py file



In our notebook



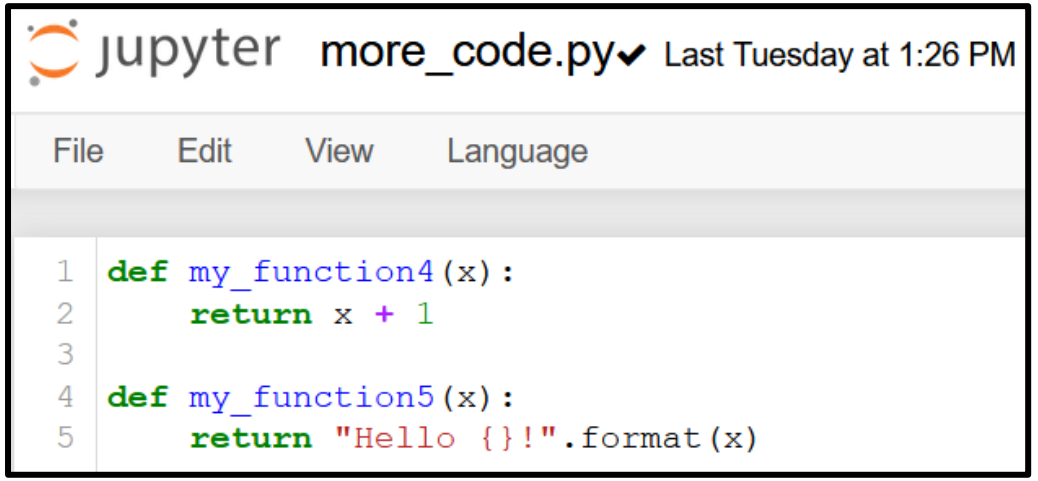
Importing an entire module

You can also use an alias for the module using the **as** keyword, as before.

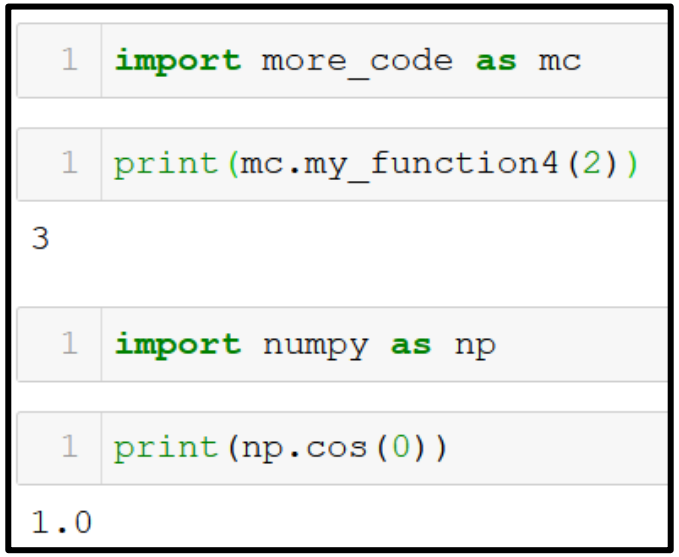
- Typically, we often do it with the Numpy library!

import numpy as np

more_code.py file



In our notebook



Folders to import from

Observation: Python imported functions from my_code.py, which was located in the same folder as my Notebook.

- **But there was no numpy.py file in this location.**
- **What happened?**

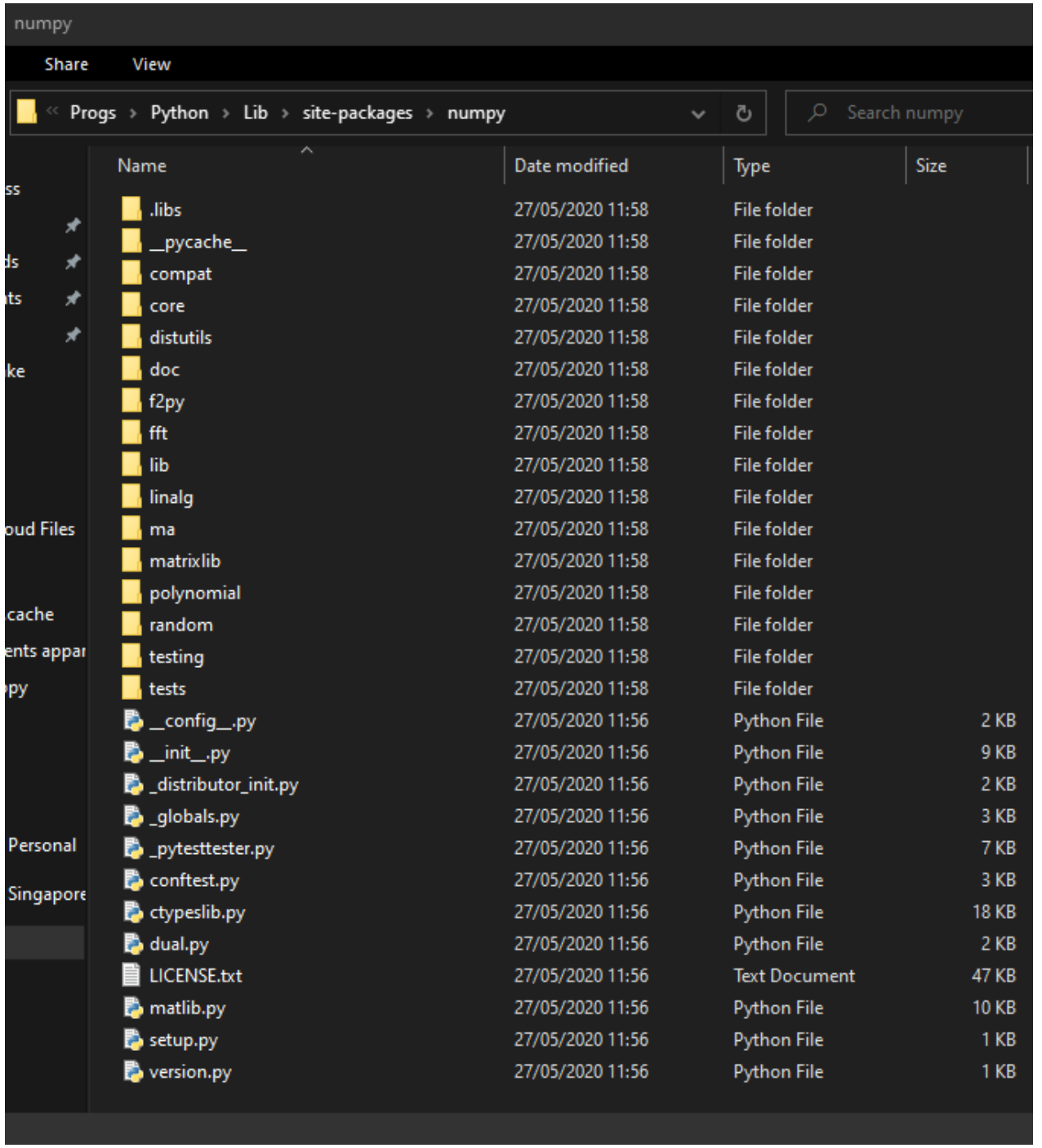
- Python looks for files in your current folder first,
- And then looks in your Python installation directory.

- When you installed the Numpy package with **pip** on Week 1, you downloaded some **numpy files** and stored them in your Python installation folder!

Folders to import from

- Python looks for files in your current folder first,
- And then looks in your Python installation directory.

- When you installed the Numpy package with **pip** on Week 1, you downloaded some **numpy** files and stored them in your Python installation folder!



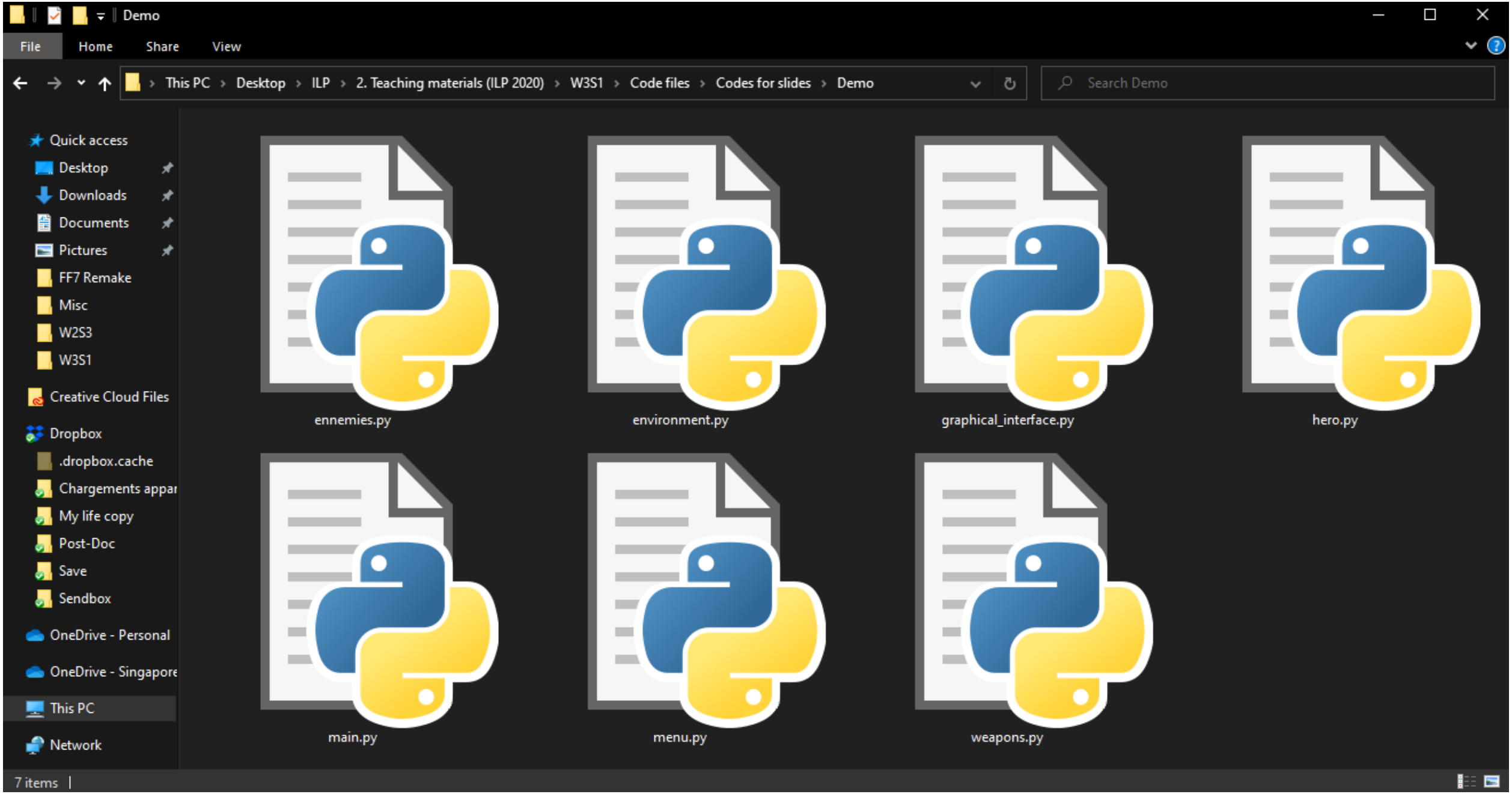
Project organizing

Question: How to organize your code/projects, when you start to have many functions working together?

- In many projects, several developers will often collaborate on a single project.
- And each one of them will end up working on a single aspect of the project.

E.g. in a video game:

- One developer can design the main character,
- Another one will develop enemies,
- Another one will develop items and weapons,
- Another one will design the map/environment in which heroes and enemies evolve,
- Etc.


Demo
File
Home
Share
View
This PC
Desktop
ILP
2. Teaching materials (ILP 2020)
W3S1
Code files
Codes for slides
Demo
Search Demo
Quick access
Desktop
Downloads
Documents
Pictures
FF7 Remake
Misc
W2S3
W3S1
Creative Cloud Files
Dropbox
.dropbox.cache
Chargements appar
My life copy
Post-Doc
Save
Sendbox
OneDrive - Personal
OneDrive - Singapore
This PC
Network
ennemies.py
environment.py
graphical_interface.py
hero.py
main.py
menu.py
weapons.py
7 items

Project organizing

In practice,

- Each developer will then work on his/her own .py file, taking care of his/her specific subtask.
- Later on, other developers might **import** functions from files created by other people.
- Eventually assemble all pieces in a **main.py** file!

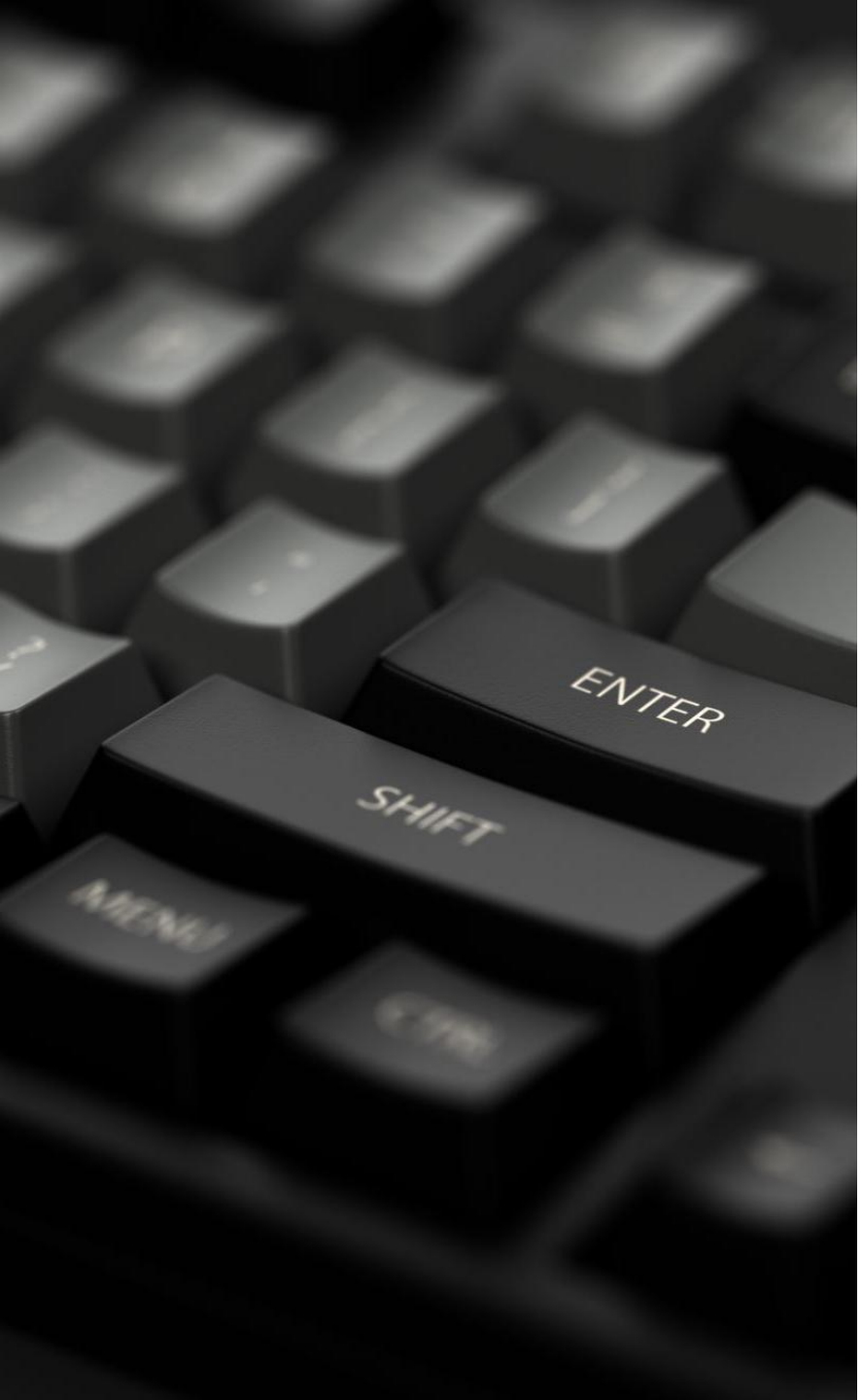
- This is something very common in programming projects.

- **Important:** If your code will be reused by other people, even though they have not coded it, you need to build the habit of documenting your functions! *(Otherwise, they will need to read your code to understand it and they will hate you for it)*

Matt's Great advice

Matt's Great Advice: Good import practices and project organizing.

1. As with the variables and functions, it is a good idea to **make your file names explicit**.
2. Have **a file for each sub-concept**, and **a single main file that assembles them all** at the end.
3. It is often better to **import only what is needed** (**from** … **import** …) rather than importing everything which is overkill. (**from** … **import** *, **import** …).
4. Comment your code! (Don't be THAT guy).



Conclusion (Chapter 12)

- What is the **import procedure** doing exactly?
- How to organize your different codes for better **project organizing**?
- **Our first mini-project**.

Let us move on to our first Mini-project now!

Wait for me to explain and show you first.